

Relatório

Leticia Bastias Fabri

Problema 1: jantar dos filósofos.

Dinâmica do problema: O jantar dos filósofos é um exemplo de concorrência que mostra a dificuldade de coordenar processos que compartilham dos mesmos recursos.

Cada um dos cinco filósofos altera seu estado entre pensar e comer – esta ação sendo possível somente quando o filosofo possui tanto o garfo da direita quanto da esquerda. O impasse surge porque os garfos estão posicionados entre dois filósofos, o que significa que dois indivíduos compartilham o mesmo talher, sem poderem usar o mesmo simultaneamente, representando uma exclusão mútua.

Esse processo ingênuo – todos os cinco filósofos pegam primeiro o garfo da esquerda, e depois esperam o da direita (que já foi pego e nunca ficará livre) – causa um deadlock. Isso porque nenhum dos filósofos irá conseguir pegar o segundo talher, e vão ficar nessa situação para sempre.

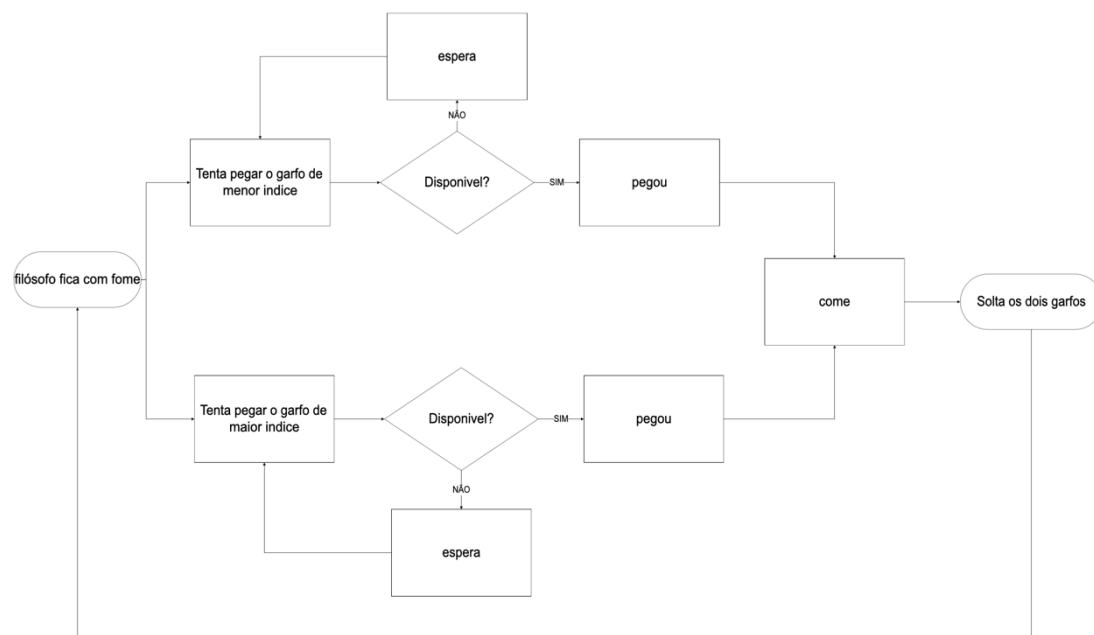
Para próxima análise, é importante definir as quatro condições de Coffman que possibilitam um deadlock:

1. **Exclusão mútua** → um garfo só pode ser usado por um filósofo por vez.
2. **Manter-e-esperar** → cada filósofo segura um garfo enquanto espera outro.
3. **Não preempção** → não dá para “arrancar” um garfo da mão de alguém.
4. **Espera circular** → filósofo 0 espera do 1, que espera do 2... fechando um ciclo.

No protocolo de antes, todas essas condições existem, por isso o deadlock acontece.

A condição que será quebrada será a da Espera circular, pois a solução escolhida é a de hierarquia de recursos – onde cada filósofo irá pegar primeiro o garfo de menor índice, e depois o de maior índice. Essa estratégia elimina a espera circular, pois agora existe uma ordem fixa sobre os garfos (recursos), assim é impossível de se criar um ciclo. Ou seja, mesmo que alguns filósofos fiquem esperando, nenhum deles ficará em deadlock.

Fluxograma do protocolo:



Problema 2 – Threads e semáforos

Cenário sem semáforos: os threads eram lançadas ao mesmo tempo (sem sincronização), tentando realizar Count++ simultaneamente. O resultado disso é um tempo de ação (throughput) pequeno, mas um contador com um valor abaixo do que deveria ter. Sabemos que o valor dele está errado, pois teoricamente, o valor final deve ser thread X incrementos. Porém, como observado no experimento abaixo, isso não ocorreu, porque como os threads vão de maneira “independente” realizar Acquire (ler valor do contador) -> Count++ -> Release (devolver o contador com novo valor),

enquanto uma está no Acquire, outra pode estar no Count++, assim gerando divergências nos valores lidos por cada thread, com incrementos sendo perdidos e valor final menor que o esperado.

```
import java.util.concurrent.*;

public class Corrida1 {
    static int count = 0;

    public static void main(String[] args) throws Exception {
        int T = 8;
        int M = 250_000; // cada thread faz M incrementos
        ExecutorService pool = Executors.newFixedThreadPool(T);

        Runnable r = () -> {
            for (int i = 0; i < M; i++) {
                count++;
            }
        };

        long t0 = System.nanoTime();
        for (int i = 0; i < T; i++) pool.submit(r);
        pool.shutdown();
        pool.awaitTermination(1, TimeUnit.MINUTES);
        long t1 = System.nanoTime();

        System.out.printf("Esperado=%d, Obtido=%d, Tempo=%.3fs%n",
            (long) T * M, count, (t1 - t0) / 1e9);
    }
}
```

```
[leticiafabri@MacBook-Air-de-Leticia-2 Desktop % java Corrida1
Esperado=2000000, Obtido=372134, Tempo=0,021s
```

Cenário com semáforos: Com a implementação de um semáforo, é possível controlar a ação e ordem de execução de cada thread, por meio de Semaphore (1, true), onde 1 é o número de thread que vai atuar por vez, e True mantém uma fila FIFO (e serve de ordem de chegada), que elimina inanição por thread.

O semáforo garante o happens-before e visibilidade ao implementar sem.acquire(), que bloqueia a thread se outra já estiver na seção crítica e sem.release(), que libera.

Assim só um thread por vez executa Count++, garantindo exclusão mútua. Além disso, essas ações garantem o happens-before e visibilidade, pois na prática, tudo que a thread A fez, será certamente lido pelo thread B subsequente.

A única desvantagem, é que o uso do semáforo perde no throughput, pois mantém os threads em uma fila, esperando até seu momento de execução.

```
import java.util.concurrent.*;

public class Semaforo {
    static int count = 0;
    static final Semaphore sem = new Semaphore(1, true); // 1 por vez
    public static void main(String[] args) throws Exception {
        int T = 8;
        int M = 250_000;
        ExecutorService pool = Executors.newFixedThreadPool(T);

        Runnable r = () -> {
            for (int i = 0; i < M; i++) {
                try {
                    sem.acquire();
                    count++;
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                } finally {
                    sem.release();
                }
            }
        };

        long t0 = System.nanoTime();
        for (int i = 0; i < T; i++) pool.submit(r);
        pool.shutdown();
        pool.awaitTermination(1, TimeUnit.MINUTES);
        long t1 = System.nanoTime();

        System.out.printf("Esperado=%d, Obtido=%d, Tempo=%.3fs\n",
            (long) T * M, count, (t1 - t0) / 1e9);
    }
}
```

```
[leticiafabri@MacBook-Air-de-Leticia-2 Desktop % java Semaforo
Esperado=2000000, Obtido=2000000, Tempo=4,565s
```

Problema 3 – Deadlock

Codigo com deadlock:

Thread 1 pega o LOCK_A, depois tenta pegar o LOCK_B

Thread 2 pega o LOCK_B, depois tenta pegar o LOCK_A

Por que deadlock?

1. Exclusão mútua: Cada lock só pode ser segurado por um thread de cada vez.
2. Manter e esperar: T1 segurou A -> espera B. T2 segurou B -> espera A
3. Não preempção: Os locks não podem ser tirados do thread à força.
4. Espera circular: T1 → precisa do lock de T2 T2 → precisa do lock de T1

```
public class Deadlock {  
  
    static final Object LOCK_A = new Object();  
    static final Object LOCK_B = new Object();  
  
    public static void main(String[] args) {  
        Thread t1 = new Thread(() -> {  
            log("T1: tentando adquirir LOCK_A");  
            synchronized (LOCK_A) {  
                log("T1: adquiriu LOCK_A");  
                dormir(200);  
  
                log("T1: tentando adquirir LOCK_B");  
                synchronized (LOCK_B) {  
                    log("T1: adquiriu LOCK_B");  
                    log("T1: concluiu trabalho.");  
                }  
            }  
        });  
  
        Thread t2 = new Thread(() -> {  
            log("T2: tentando adquirir LOCK_B");  
            synchronized (LOCK_B) {  
                log("T2: adquiriu LOCK_B");  
                dormir(200);  
  
                log("T2: tentando adquirir LOCK_A");  
                synchronized (LOCK_A) {  
                    log("T2: adquiriu LOCK_A");  
                    log("T2: concluiu trabalho");  
                }  
            }  
        });  
  
        t1.start();  
        t2.start();  
    }  
}
```

```
leticiafabri@MacBook-Air-de-Leticia-2 Desktop % java Deadlock
T1: tentando adquirir LOCK_A
T2: tentando adquirir LOCK_B
T1: adquiriu LOCK_A
T2: adquiriu LOCK_B
T2: tentando adquirir LOCK_A
T1: tentando adquirir LOCK_B
```

Código corrigido:

Utilizamos hierarquia de recursos (igual no problema dos filósofos) para quebrar a condição de espera circular, e impedir o deadlock. Os threads pegam primeiro o LOCK_A, e depois pegam o LOCK_B.

```
public class Corrigido {

    static final Object LOCK_A = new Object();
    static final Object LOCK_B = new Object();

    public static void main(String[] args) {
        Thread t1 = new Thread(() -> trabalho(), "T1");
        Thread t2 = new Thread(() -> trabalho(), "T2");

        t1.start();
        t2.start();
    }

    static void trabalho() {
        log("tentando adquirir LOCK_A");
        synchronized (LOCK_A) {
            log("adquiriu LOCK_A");
            dormir(50);

            log("tentando adquirir LOCK_B");
            synchronized (LOCK_B) {
                log("adquiriu LOCK_B");
                log("concluiu trabalho");
            }

            log("liberou LOCK_B");
        }
        log("liberou LOCK_A");
    }

    static void dormir(long ms) {
        try { Thread.sleep(ms); }
        catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

```
[leticiafabri@MacBook-Air-de-Leticia-2 desktop % javac Corrigido.java
[leticiafabri@MacBook-Air-de-Leticia-2 desktop % java Corrigido
T1 - tentando adquirir LOCK_A
T2 - tentando adquirir LOCK_A
T1 - adquiriu LOCK_A
T1 - tentando adquirir LOCK_B
T1 - adquiriu LOCK_B
T1 - concluiu trabalho
T1 - liberou LOCK_B
T1 - liberou LOCK_A
T2 - adquiriu LOCK_A
T2 - tentando adquirir LOCK_B
T2 - adquiriu LOCK_B
T2 - concluiu trabalho
T2 - liberou LOCK_B
T2 - liberou LOCK_A
leticiafabri@MacBook-Air-de-Leticia-2 desktop % █
```